

mycode: A Personalized, Zero-Cost Local Coding Assistant

with RAG-Augmented Generation for Embedded ML Development

Jordi Angel
Owell, Ecuador
github.com/jordiangel

May 2026

Abstract

The computational cost of frontier AI models continues to grow exponentially, placing powerful coding assistance out of reach for resource-constrained environments and offline deployments. I present **mycode**, a terminal-based personal coding assistant that runs entirely locally at zero ongoing cost, designed specifically for embedded machine learning development at Owell, a startup operating in Ecuador. mycode combines DeepSeek-Coder-V2 16B, a Mixture-of-Experts architecture with only 2.4B active parameters, with a Retrieval-Augmented Generation pipeline backed by PostgreSQL with pgvector, a personal code snippet store, and a corpus of 34 domain-specific ML and embedded systems papers totaling 2,612 embedded chunks. Empirical evaluation on a custom C++17 benchmark covering algorithms, embedded systems, and machine learning shows that RAG-based personalization improves pass rate from 60 percent to 80 percent over the bare model, closing roughly half the gap to Claude Sonnet 4.6, which achieved 100 percent on the same benchmark. On a project-scale task requiring a complete INT8 neural network inference engine in C++17, mycode matched Sonnet 4.6 on compilation and code quality metrics, with inference throughput of 100,000 forward passes in 2.75 milliseconds.

1 Introduction

The landscape of AI-assisted software development has been dominated by cloud-hosted frontier models. GitHub Copilot, Claude Code, and OpenAI Codex deliver high-quality code generation but impose ongoing subscription costs, require reliable internet connectivity, and offer no mechanism for personalization to an individual developer’s patterns. For engineers working in bandwidth-constrained or offline environments, such as embedded systems development in Ecuador, these tools are often impractical.

Simultaneously, the compute required to train and serve frontier models continues to scale aggressively. GPT-4 class models are estimated to require thousands of A100 GPU-hours for a single inference pass at scale. The trend toward ever-larger dense models exacerbates the gap between what frontier labs can deploy and what a solo developer can run locally. Mixture-of-Experts architectures have emerged as a promising counterpoint to this trend, activating only a fraction of total parameters per forward pass and enabling near-frontier performance at a fraction of the compute cost.

mycode was built to address a concrete engineering need at Owell, a startup developing embedded ML systems in Ecuador. The system must function offline, at zero marginal cost per query, and improve over time as the developer accumulates domain-specific solutions. This paper describes the architecture, implementation

decisions, evaluation methodology, and benchmarking results of mycode, including a direct comparison against Claude Sonnet 4.6, a state-of-the-art frontier model.

2 Background and Motivation

2.1 The Compute Scaling Problem

Frontier model compute has followed a consistent scaling trajectory. Training compute for leading models has roughly doubled every six to twelve months since 2018. Inference costs scale proportionally with model size for dense architectures. A 70B parameter dense model requires approximately 140GB of GPU memory for full-precision inference, far exceeding the capacity of consumer hardware.

For embedded ML development specifically, the mismatch is acute. Engineers working on quantization, real-time inference, and hardware-constrained deployment need coding assistance that understands INT8 arithmetic, fixed-point representations, memory layout, and RTOS scheduling. Frontier models provide this knowledge but at the cost of cloud dependency.

2.2 Mixture-of-Experts as a Local Solution

MoE architectures route each token through a small subset of expert networks rather than the full model. DeepSeek-Coder-V2 16B activates only 2.4B parameters per forward pass despite having 16B total parameters. This produces inference throughput comparable to a 2-4B dense model while retaining the representational capacity of a 16B model. On an Apple M3 Pro with 18GB unified memory, this translates to approximately 67 tokens per second, sufficient for interactive use.

2.3 Retrieval-Augmented Generation

RAG systems augment LLM generation by prepending retrieved context from an external knowledge store to the model prompt. The retrieval step uses semantic similarity between an

embedded query vector and a database of pre-embedded document chunks, enabling the model to reference domain-specific material not present in its training data. For a personal coding assistant, this enables two distinct use cases: retrieval of the developer’s own past solutions, and retrieval of relevant technical literature.

3 System Architecture

3.1 Overview

mycode consists of five layers: a CLI interface, an LLM inference layer, a context gathering layer, a RAG pipeline, and a persistent knowledge store. Figure 1 illustrates the data flow.

Listing 1: mycode query flow

```
User query
-> Context gathering (git diff
    , file)
-> RAG retrieval (pgvector
    similarity)
-> Conditional prompt
    injection
-> DeepSeek-Coder-V2 16B (
    Ollama)
-> Streamed response
-> Quality gate validation
-> Accept/reject save flow
-> pgvector + Obsidian vault
```

3.2 Model Layer

The model is DeepSeek-Coder-V2-Lite-Instruct, a 16B MoE model with 2.4B active parameters and 128K context length, served via Ollama on an Apple MacBook Pro M3 Pro with 18GB unified memory. Measured performance on this hardware shows a time-to-first-token of 99 milliseconds, a generation rate of 67 tokens per second, and a prompt evaluation rate of 121 tokens per second. Model cold load requires approximately 6 seconds. All subsequent queries within the same session incur no reload overhead.

3.3 Embedding and Vector Store

Text embeddings are generated using all-MiniLM-L6-v2 from the sentence-

transformers library, producing 384-dimensional vectors. This model was selected for its low latency on CPU, requiring no GPU for embedding inference. Embeddings are stored in PostgreSQL 16 with the pgvector extension, using an IVFFlat index with cosine distance for approximate nearest neighbor search. Two tables are maintained: `code_snippets` for the developer’s accepted solutions, and `resources` for paper chunks.

3.4 Knowledge Corpus

The knowledge corpus consists of 34 domain-specific documents spanning quantization methods, transformer architectures, embedded systems programming, real-time operating systems, and linear algebra. Documents were converted from PDF to markdown and segmented into 500-word chunks, yielding 2,612 total chunks. Key documents include foundational papers on quantization, specifically GPTQ, AWQ, LLM.int8, LoRA, QLoRA, and mixed-precision training, as well as reference texts on FreeRTOS, real-time C++, and embedded systems programming. All documents were selected for direct relevance to the embedded ML development work at Owell.

3.5 RAG Pipeline Design

A critical design decision was made regarding when to inject retrieved context into the model prompt versus presenting it as a reference panel to the user. Early evaluation revealed that injecting paper chunks into the prompt consistently degraded generation quality. The root cause was that prose descriptions of technical concepts from papers, when injected as prior context, caused the model to reproduce patterns from the retrieved text rather than generating clean implementations. In one documented case, a retrieved code snippet containing a bug (`int8_t::max` instead of `INT8_MAX`) was reproduced faithfully by the model in three consecutive generation attempts, despite explicit instructions to the contrary.

The final pipeline implements a selective injection policy. Code snippets from `code_snippets` with cosine similarity above 0.55 are injected into

the prompt, as these represent the developer’s own validated solutions to similar problems. All paper chunk matches, regardless of similarity score, are displayed to the user as a reference panel after generation but are never injected into the model prompt. This preserves model generation quality while surfacing relevant literature for the developer’s reference.

3.6 Quality Gate

A compilation-based quality gate was added to the save flow after a buggy snippet was found to have propagated through the pipeline. Before any code snippet is persisted to the database or Obsidian vault, the system validates it by syntax-checking Python via `py_compile` or syntax-checking C++ via `g++ -fsyntax-only`. Code that fails validation is presented to the user with the compiler error and requires explicit confirmation to save. This prevents the knowledge base from accumulating incorrect patterns.

3.7 CLI and Save Flow

The CLI is implemented in Python using Click and Rich, invoked via a bash wrapper at `/usr/local/bin/mycode`. After each generation, the user is presented with an accept or reject prompt. Accepted solutions are embedded, stored in `code_snippets` with metadata including language, category, and project, and saved as structured markdown to an Obsidian vault organized in a hybrid topic and project hierarchy.

4 Evaluation Methodology

4.1 Benchmark Design

Standard coding benchmarks such as HumanEval and MBPP evaluate Python function generation on general-purpose problems. These are insufficient for evaluating a system designed for embedded ML development in C++. I designed a custom evaluation suite with three components.

The first component is a 10-problem C++17 benchmark covering the three primary domains of mycode usage: algorithms, embedded systems,

and machine learning. Problems include binary search, linked list reversal, merge sort, circular buffer implementation, bitwise register manipulation, INT8 quantization, matrix multiplication, ReLU activation, stack from queues, and Q16.16 fixed-point multiplication. Each problem is evaluated by compiling the generated code against a hand-written test driver and checking for correct output. The metric is pass at one, meaning a single generation attempt per problem.

The second component is a project-scale evaluation requiring the model to generate a complete, self-contained C++17 program implementing a lightweight neural network inference engine for embedded deployment. Requirements include INT8 weight storage, INT32 accumulation, ReLU and sigmoid activations, a two-layer feed-forward network with architecture Linear(2,4), ReLU, Linear(4,1), Sigmoid, no heap allocation in the forward pass, and a benchmark measuring 100,000 forward pass calls. Scoring covers compilation, functional correctness on XOR, code quality checks, and inference throughput.

The third component is a RAG quality benchmark measuring retrieval precision across ten domain-specific queries, evaluating embedding score distributions, pipeline latency, and the fraction of retrieved chunks scoring above the injection threshold.

4.2 Comparison Baseline

Claude Sonnet 4.6 was used as the frontier model baseline. Sonnet 4.6 was queried with identical problem specifications through the Claude web interface. Generated code was saved locally and compiled using the same test drivers and evaluation scripts used for mycode. This ensures a fair comparison on functional correctness independent of any differences in output formatting.

5 Results

5.1 Hardware Benchmarks

Table 1 summarizes the inference performance of mycode on the M3 Pro.

Table 1: Inference performance on Apple M3 Pro, 18GB.

Metric	Value
Model cold load	6.0 seconds
Time to first token	99 milliseconds
Prompt eval rate	121 tokens per second
Generation rate	67 tokens per second
RAG retrieval latency, avg	194 milliseconds
RAG retrieval latency, min	95 milliseconds
RAG retrieval latency, max	428 milliseconds

5.2 RAG Retrieval Quality

Table 2 summarizes embedding score distributions across ten domain-specific queries.

Table 2: RAG retrieval score distributions across ten queries.

Metric	Resources	Code Snippets
Min similarity	0.238	0.000
Max similarity	0.694	0.515
Avg similarity	0.506	0.161
Above threshold	26 of 30	5 of 30

The low average similarity for code snippets reflects the small size of the snippet store at evaluation time, six total entries, rather than poor retrieval quality. As the developer accumulates solutions through normal usage, this distribution is expected to improve substantially.

5.3 C++ Benchmark Results

Table 3 presents pass rates on the 10-problem C++17 benchmark.

RAG personalization improved pass rate by 20 percentage points over the bare model, from 60 percent to 80 percent. mycode with RAG closes approximately half the gap to Sonnet 4.6. Average generation latency with RAG is marginally lower than bare generation due to caching effects in the embedding model.

Table 3: C++17 benchmark pass rates.

Domain	DS bare	mycode	Sonnet
Algorithms	2/4	3/4	4/4
Embedded	2/4	3/4	4/4
ML	2/2	2/2	2/2
Total	6/10	8/10	10/10
Pass rate	60%	80%	100%
Gen time	3.5s	3.3s	2s
Cost/query	\$0	\$0	\$0.01

5.4 Project-Scale Evaluation

Table 4 presents results on the neural network inference engine project task.

Table 4: Project-scale evaluation results.

Metric	mycode	Sonnet 4.6
Compiled	Yes	Yes
XOR functional	No	Partial
Attempts required	2 of 3	1 of 1
INT8 weights	Yes	Yes
INT32 accumulation	Yes	Yes
No heap in forward	Yes	Yes
Activation functions	Yes	Yes
Benchmark present	Yes	Yes
Correct entry point	Yes	Yes
100k forward passes	2.75 ms	1.98 ms
Quality score	68 of 100	78 of 100

Both systems compiled successfully and passed all six code quality checks. Neither system achieved fully correct XOR classification. mycode produced outputs of 0.5 for all four inputs, indicating the forward pass computation collapsed to a constant. Sonnet 4.6 correctly classified two of four inputs but produced 0.519 for inputs (0,0) and (1,1), which lie above the 0.5 decision boundary and were therefore classified incorrectly. On inference throughput, mycode achieved 2.75 milliseconds for 100,000 forward passes compared to 1.98 milliseconds for Sonnet 4.6, a difference of 0.77 milliseconds attributable to differences in generated implementation style rather than hardware.

6 Analysis

6.1 When RAG Helps and When It Hurts

The evaluation revealed a consistent pattern. RAG improves performance on targeted retrieval tasks where the developer has previously solved a similar problem. On the binary search benchmark, the developer’s saved solution scored 0.68 cosine similarity and was correctly injected, allowing the model to reference a validated implementation. This pattern held across the algorithm and embedded domains where prior solutions existed.

RAG consistently degraded performance on generative tasks requiring novel composition, particularly when retrieved paper chunks were injected into the prompt. The selective injection policy, restricting injection to code snippets above 0.55 similarity and excluding all paper chunks from prompt injection, resolved this degradation and produced the 80 percent pass rate.

6.2 Quality Gate Impact

The quality gate prevented one confirmed case of bug propagation. A saved INT8 quantization snippet containing `int8_t::max`, which is not valid C++ syntax, was retrieved with 0.43 similarity on every quantization-related query and caused the RAG-augmented model to reproduce the same compile error across three consecutive generation attempts on the project task. Deleting the buggy snippet and enabling the compilation-based quality gate eliminated this failure mode entirely.

6.3 Deployment Context

mycode was designed for a specific operational context. Owell is developing embedded ML systems in Ecuador, where internet connectivity may be unreliable and cloud API costs accumulate at the scale of a small startup. Running inference locally on an M3 Pro eliminates per-query costs entirely. The system operates fully offline after initial model download. The learning loop, wherein accepted solutions are embedded and

stored for future retrieval, means the system becomes more useful over time as domain-specific solutions accumulate.

6.4 Limitations

Several limitations of the current system are worth noting. The embedding model, `all-MiniLM-L6-v2`, is a general-purpose model not fine-tuned on technical or domain-specific text. Cosine similarity scores for technical queries ranged from 0.24 to 0.69, lower than one might expect from a domain-adapted model. Replacing it with a code-aware embedding model such as `nomic-embed-text` may improve retrieval precision. The code snippet store contained only six entries at evaluation time, limiting the impact of the learning loop. With sustained usage over weeks or months, the retrieval benefit is expected to grow substantially. Finally, the XOR project task exposed a known limitation of locally quantized models on multi-step compositional reasoning, where generating correct numerical constants requires precise arithmetic that 2.4B active parameter models may not reliably perform.

7 Related Work

DeepSeek-Coder-V2 achieves 90.2 percent on HumanEval and 76.2 percent on MBPP Plus for the full 236B variant, and 81.1 percent on HumanEval for the 16B Lite variant used in mycode. These scores were achieved on Python-centric benchmarks. The custom C++17 benchmark used in this work is more representative of embedded ML development tasks and shows a 60 percent bare pass rate for the 16B Lite model, consistent with the expectation that domain-specific evaluations yield lower scores than general benchmarks.

RAG systems have been shown to reduce hallucination rates and improve factual grounding in generation tasks. Research from Stanford indicates that poorly evaluated RAG systems can produce hallucinations in up to 40 percent of responses despite accessing correct information. The quality gate and selective injection policy in mycode address this failure mode directly.

MoE architectures, as described in the DeepSeek-V2 technical report, offer strong local performance within memory constraints relative to dense models of comparable parameter count. The 16B MoE model with 2.4B active parameters achieves generation rates competitive with dense 7B models while retaining the knowledge capacity of a 16B model.

8 Future Work

Several directions are planned for continued development of mycode. First, replacing `all-MiniLM-L6-v2` with a domain-adapted embedding model such as `nomic-embed-text` is expected to improve retrieval precision on technical queries and reduce the sensitivity of the similarity threshold. Second, a query classifier to detect project-scale generative tasks and automatically disable RAG injection would remove the need for manual configuration. Third, as the Owell codebase grows, project-scoped retrieval limiting search to solutions from the current project context will improve relevance. Fourth, lightweight LoRA fine-tuning on the accumulated solution corpus is a longer-term direction that could further personalize the model beyond what RAG alone provides. Finally, integrating a continuous benchmark harness that re-evaluates the system on the C++17 test suite after each session would provide ongoing visibility into system quality as the knowledge base grows.

9 Conclusion

I presented mycode, a personalized local coding assistant combining DeepSeek-Coder-V2 16B MoE with a RAG pipeline backed by pgvector and a domain-specific corpus of 34 embedded ML documents. Empirical evaluation on a custom C++17 benchmark shows that RAG-based personalization improves pass rate from 60 percent to 80 percent over the bare model, a 20 percentage point improvement achieved at zero additional compute cost. On a project-scale neural network inference engine task, mycode matched Claude Sonnet 4.6 on compilation and code quality while

running entirely offline at zero marginal cost per query.

The key engineering insights from this work are three. First, RAG injection must be selective: injecting paper chunks into the prompt degrades generation quality, while injecting high-confidence matches from the developer’s own validated solutions improves it. Second, knowledge base quality matters more than quantity: a single buggy snippet caused consistent downstream failures across multiple queries, motivating the compilation-based quality gate. Third, MoE architectures make locally competitive coding assistance genuinely feasible on consumer hardware, closing roughly half the performance gap to frontier models at a fraction of the operational cost.

This paper is available at <https://github.com/jangel19/jordiai>.